

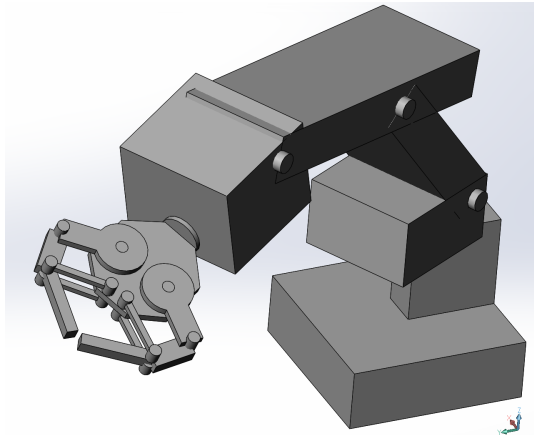
Rigid Body Control Via Muscle Activity Interpretation

1st Reed Evans
Dept. of Electrical Engineering
Stanford University
reedev@stanford.edu

2st Polo Contreras
Dept. of Electrical Engineering
Stanford University
jcontr83@stanford.edu

3rd Preston Rogers
Dept. of Electrical Engineering
Stanford University
parogers@stanford.edu

I. INTRODUCTION



Traditional interfaces can be a limiting factor in some applications of robotics: to name a few, exploration of extreme environments, robotics for telepresence and handling of hazardous materials. Although traditional interfaces like handheld controllers can be used for real-time control of robots, they have inherent limitations, and can be unintuitive for users who are not familiar with them already. Haptic interfaces allow for more detailed movements and are more intuitive for users, but suffer from high hardware cost and complexity. A technology that can exploit movements intuitive to users, with low cost and mechanical simplicity, is to use electrodes attached to the user, to detect muscle movements [3]. There is inherent ambiguity in these signals, however, and the data needs to be processed in order to interpret them as clear instructions. Machine learning provides us with tools that we can use to interpret signals generated by muscle activation, allowing us to train a system to interpret electricity indicating the activation of one or more muscles and allowing us to use the electrodes for real-time control of a (real or virtual) device. With a sufficiently robust system in place that could relate the movements of a user's limb to device instructions, the movements of a person can be translated in real time to the movements of a robot or another application that the user intends to control.

In this project, the inputs are voltages generated by electromyographic sensors attached to a user's arm, while the

outputs are predicted movements of the arm. Although this is therefore a supervised learning problem, both the data from the muscle sensors and the data from the accelerometer (used to label the training and test data, as we will detail further in this report) required a certain amount of preprocessing.

Finally, these same outputs were applied to a model of a robotic arm, capable of moving freely in a two-dimensional task space. Adapted from a physical robot arm with five degrees of freedom, the robot arm has three parallel revolute joints, and the movements of the user's hand are associated with movements of its end effector. A sequence of these movements can be transformed into a sequence of settings of the joints in the robot arm, and the robot arm can thus reproduce the movements of the user's arm despite the morphological differences between the two.

II. RELATED WORKS

Noninvasive control of robot arm using EEG data: In [6], the authors deal with a similar application to our own, using electroencephalographic (EEG) signals for control of a robotic arm with an end effector in a 2D plane. The use of a brain-computer interface (BCI), however, means the input data and hardware for its collection are entirely different, with a more specific focus on medical applications (patients with neurological disorders).

Using thresholding to open and close fingers using EMG data: In our implementation, we used EMG data to determine the desired motion of a robot arm at a particular time instant. Similarly, [11] used EMG data to mimic arm movements performed by the user. Unlike our implementation, the motions being mimicked in this application were finger movements so the EMG sensors were primarily needed for detecting motions in the tendons whereas our project primarily measured muscle movements.

EEG Signal Pre-processing: In [8], they use a sliding window for the pre-processing step akin to the windowing method we incorporate. However, they do not use the sliding window in conjunction with signals that are amicable to direct classification but instead use Linear Discriminant Analysis to find a sub-space where the true EEG data is easier to classify.

Rigid Body Inverse Kinematics: [9] uses inverse kinematics to find out the four joint angles ($q_1 \dots q_4$) during the training phase in a way similar to our method where the cartesian

coordinates of particular points of the arm are declared, and the established functions compute the angles necessary. Though the angles in their method can be solved for analytically as the motion sensors establish a larger number of constraints versus the single constraint we establish regarding the desired end-effector cartesian coordinate.

Kernelization of Binary Classifiers: [10] used SVM to classify sEMG data (electrical muscle data) into several categories that corresponded to various hand positions. While doing so, they compared the accuracy a linear model and an RBF model. They found that the RBF model outperformed the linear model by an average of over five percent. This finding relates to our implementation insofar as we used the RBF kernel for the perceptron algorithm after discovering that the do-nothing kernel (i.e., the data multiplied by the identity matrix) did not yield accurate results when trying to classify data as neutral versus motion.

III. DATASET

To simplify the problem, we divide the possible directions of movement into five categories: up, down, forwards, backwards and neutral (no movement). We collected our own data using surface electromyographic (sEMG) muscle sensors and accelerometers attached to the arm of a team member (Polo Contreras), where the acceleration data provides ground truth for which direction the arm is moving. A muscle sensor was each placed at the forearm, bicep, tricep, and shoulder for a total of four muscle sensors. The accelerometers were placed at the back of the hand, and to ensure the clarity of the muscle signals an 8-pound weight was attached to the hand during trials. All sensors reported at 60 Hz.

Accelerometer data was received as two voltage signals in the range $[0V, 3.3V]$, each representing acceleration in a given axis. We preprocess the acceleration data by using a 100-sample rolling average for smoothing and noise reduction; by subtracting the instantaneous accelerometer values from the respective average values, we generate the samples we will use as ground truth. All samples are then mapped to the range $[-1, 1]$ in each axis. To label our acceleration data into the five directions listed above, we employ the K -means clustering algorithm with five hard-coded initial centroids. This gave very good labels that could be used for training and testing, shown in Figure 1. To reduce the influence of noise factors, we trained and tested on data collected in the same session. The samples were randomly shuffled and then split into a training set and test set, both of equal size (6000 samples each).

Preprocessing of the input data was also necessary for accurate prediction. The muscle sensors themselves, via internal firmware, register the electrical signals generated by muscles and rectify and integrate them so they can easily be read by a microcontroller. These filtered signals are captured as voltage values, and after we record them we process each one of the signals in the same way: the derivative of each signal is approximated by performing a subtraction between the current voltage value and the preceding voltage value, the absolute value of the approximate derivative is also calculated and

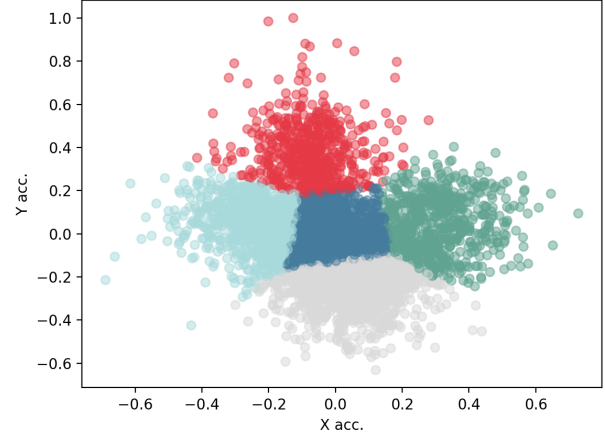


Fig. 1. Preprocessed training data labeled into five class using K -means clustering. Red is up, grey is down, green is forward, light green is backward, and blue is neutral.

recorded, a rolling average value is calculated with the most recent 100 voltage values and recorded, and the difference between the immediate voltage value and the rolling average value is recorded as well. This resulted in a total of 20 channels of input data, and our predictions were most accurate when using all of them at once (as opposed to when any set of four channels, representing a type of data for all four sensors, was removed).

There are a wide variety of potential noise factors in developing this application. While the number of degrees of freedom in a limb necessary to move an object freely in three-dimensional space is six, the human arm has seven; it is therefore possible to achieve the same movements of the hand with multiple postures of the arm. Additionally, despite the restricted types of movements being registered by the accelerometer, the number of muscles in the arm used to achieve these movements is much greater than the number of available sensors: there are more than twenty muscles in the human arm. Additionally, between different recordings, the placement of electrodes on the arm may vary slightly, as may the intensity of signals when the sensors are placed on different users.

IV. METHODS

The purpose of this project is to control a robot arm via sensor inputs. We wanted to make the control scheme intuitive. To this end, we decided upon having the control inputs translate to end-effector movements in cartesian space. This requires some method of converting cartesian coordinates of the end-effector to angle measurements for the joints, a process called inverse kinematics. There are numerous ways to accomplish this. For example, one can represent the new, desired end-effector position in the configuration space (i.e., angle values $(\theta_1, \dots, \theta_n)$ where n is the number of joints) that include sampling of the configuration space and running

one of a number of search methods, such as A*, to find a possible joint path. However, these techniques are often computationally expensive and require a goal projection onto the valid configuration space [2]. Additionally, many techniques are not amicable to a 5-degrees of freedom robot arm due to interpolation issues in task-space [1]. As such, we chose to ease computation by decomposing arm movements to motions within a 2-dimensional sub-space of $\mathbb{R}^3$ and rotations of the base. To do so, we use a technique that was highlighted in the first lecture notes as a technique for function maximization. Although used for function maximization, this function is quite versatile as it is an iterative way to find the zeroes of a function. This method is referred to as the Newton Method, and for our application we use it to find the zeroes of the following function:

$$\begin{aligned}\theta &\equiv [\theta_1 \quad \theta_2 \quad \theta_3]^T \\ g(\theta) &\equiv \begin{bmatrix} -l_1 \sin(\theta_1) + l_2 \cos(\theta_2 + \theta_1) + l_3 \cos(\theta_3 + \theta_2 + \theta_1) \\ l_1 \cos(\theta_1) + l_2 \sin(\theta_2 + \theta_1) + l_3 \sin(\theta_3 + \theta_2 + \theta_1) \end{bmatrix} \\ f(\theta) &= g(\theta) - \begin{bmatrix} x_{\text{end}} \\ y_{\text{end}} \end{bmatrix}\end{aligned}$$

The function $g(\theta)$ simply relates the joint positions to the end-effector's position using angles with respect to the previous link's position in the kinematic chain. Truly, this is not the true position of the end-effector as the end-effector exists in $\mathbb{R}^3$, but is the position with respect to the plane defined by the joint angle of the base link. This description of $g(\theta)$ intuitively why $f(\theta) = 0$ yields the angles we desire. Namely, when $f(\theta) = 0$ the desired 2-D position $(x_{\text{end}}, y_{\text{end}})$ is achieved. Without difficulty, the first order approximation to this function can be found and we are able to find the zeroes of this linearly approximated function with the following iterative update...

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - J(\theta^{(t)})^\dagger \left(f(\theta^{(t)}) \right)$$

...where $J(\theta^{(t)})$ is the Jacobian, evaluated at the previous iterate, that relates the angle measurements to the 2-D end-effector position and $J(\theta^{(t)})^\dagger$ is simply the pseudo-inverse of this Jacobian.

Once the mapping of goal cartesian coordinate positions to angles was established, we shifted focus to how to link sensor values to goal cartesian coordinate positions. We chose to keep this relation relatively simple for initial testing by limiting the movements of the end-effector to up, down, forward, and backward so that it could be framed as a classification problem. Doing so would allow us to use methods shown in class that involved multi-class discrete labelling, namely softmax regression [5]. Softmax regression assumes that the probability of y given η takes the following form:

$$p(y = k \mid \eta) = \frac{e^{\eta_k}}{\sum_j e^{\eta_j}}$$

Where, for our use, η_j is simply $\theta_j^T x^{(i)}$. The likelihood is then the following:

$$\ell(\theta) = \prod_i \frac{e^{(\theta_{y^{(i)}}^T x^{(i)})}}{\sum_j e^{(\theta_j^T x^{(i)})}}$$

Where $y^{(i)}$ is the label for the i th data-point.

Using the log-likelihood as the objective function, the gradient, vectorized gradient, and update rule used for gradient ascent for softmax regression are shown below.

$$\begin{aligned}m(\theta) &\equiv \log \ell(\theta) \\ \nabla_{\theta_f} m(\theta) &= \sum_{i=1}^n \sum_{l=1}^k 1\{y^{(i)} = l\} \left(\{f = l\} x^{(i)} - \frac{x^{(i)} e^{\theta_f^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right) \\ \nabla_{\theta} m(\theta) &= X^T (Y' - \text{diag}^{-1}(e^{X\theta} \mathbf{1}) e^{X\theta}) \\ \text{Where exponential is performed element-wise} \\ \theta &:= \theta + \alpha \nabla_{\theta} m(\theta)\end{aligned}$$

For the vectorized gradient, Y' is an $n \times k$ matrix where $Y'_{i,j}$ is 1 if $y^{(i)} = j$ and 0 otherwise.

One problem remains when using the softmax classifier on our labeled data – the five classes do not meet at a single point. The neutral class sits within the others and therefore softmax will not be able to label points of this class; this is shown in Figure 2. To solve this problem we use the perceptron algorithm with a radial basis function (RBF) kernel alongside the softmax classifier. The RBF perceptron method is effective at classifying clusters of data surrounded by other data, which makes it well suited to classify the neutral class here. To make a prediction, the RBF perceptron predicts if it is the neutral class first. If it is then that is its final label. If not, then softmax is used to determine the label from among the four remaining classes. A question of performance comes up when using the perceptron algorithm in this application. Since it uses all the training data when making predictions the prediction time can be slower than other methods, which is a concern for real-time control. We measured the prediction time during out tests and found that the perceptron algorithm was still able to make predictions well above 60 Hz and therefore can make predictions as fast as our data comes in.

Thresholding acceleration values is a natural way of classifying the data, but a human may be overly safe and set these values in such a way as to cause many false negatives. We then tried ML techniques, particularly Mixture of Gaussians and K-means, for the classification of the training data. Mixture of Gaussians falls under the realm of unsupervised learning. It assumes there are k latent classifications and that the data follows a multivariate Gaussian distribution with parameters that depend on the latent classification.

For our application, we further assumed that the varying Gaussian distributions the accelerometer data came from had equivalent covariances. The parameters for the distributions

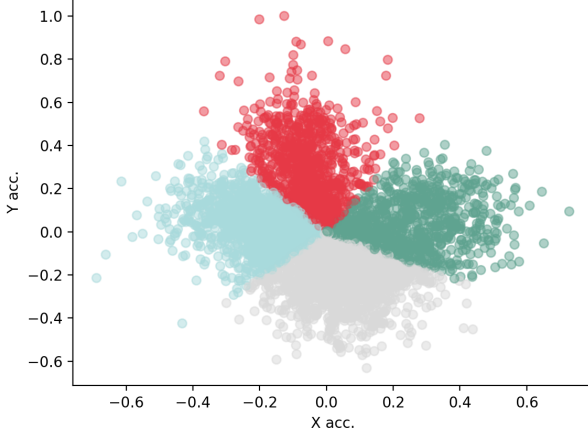


Fig. 2. This shows the result of running the softmax classifier on the acceleration data itself. Note how the neutral class in the middle is completely gone.

are found via a maximizing the log of the probability of the data where this is found as follows:

$$\begin{aligned}
 p(y) &= \phi^y (1 - \phi)^{1-y} \\
 p(x | y = 0) &= \mathcal{N}(x; \mu_0, \Sigma) \\
 p(x | y = 1) &= \mathcal{N}(x; \mu_1, \Sigma) \\
 \ell(\theta) &= \sum_i p(x^{(i)}) = \sum_i \sum_y p(y) p(x | y) \\
 m(\theta) &= \log \ell(\theta)
 \end{aligned}$$

Where $\mathcal{N}(\cdot; \mu, \Sigma)$ is the multivariate Gaussian distribution and $\ell(\theta)$ is established by the law of total probability

After GMM produced unfavorable results, K-means was chosen as it can determine highly optimal data associations.

The function to be minimized for K-means is the distortion function given as follows:

$$J(c, \mu) = \sum_{i=1}^n \|x^{(i)} - \mu_{c(i)}\|^2$$

Where $c^{(i)}$ is the cluster the i th data-point is associated with and $\mu_{c(i)}$ is the mean of said cluster.

This is, unfortunately, a non-convex, combinatorial problem that cannot be reliably solved to optimality. As a compromise, we minimize it component-wise by repeatedly performing the following two steps until convergence:

- 1) $c^{(i)} := \operatorname{argmin}_j \|x^{(i)} - \mu_j\|^2$
- 2) $\mu_j := \frac{\sum_i \mathbb{1}_{\{c^{(i)}=j\}} x^{(i)}}{\sum_i \mathbb{1}_{\{c^{(i)}=j\}}}$

Applying K-means with minimal pre-processing yielded labels quite comparable to the classifications given by a thresholding scheme that incorporated hard-coded values found by inspection.

To gain a sense of the effectiveness of our application of soft-max, we optimized a three-layer neural network. Each hidden layer incorporated 128 hidden units and the final layer

has five units corresponding to probabilities for each of the five classes. The weights were optimized using the Adam optimizer; Adam incorporates current and past gradients and weights them via a method akin to a moving average. The gradient at a particular iteration is found using backpropagation; a method of gradient calculation that takes advantage of the layered nature of neural networks. The nonlinear activation function for each hidden layer was the standard Rectified Linear Unit (ReLU) function:

$$h(x) = \max\{x, 0\}$$

V. RESULTS AND DISCUSSION

Early on, the assumptions made for the Mixture of Gaussians proved unjustified as the accelerometer data did not follow Gaussian distributions. Instead, conceptually, there existed a center grouping of data points that denoted the neutral state and four surrounding groupings that denoted the user's intended movements where y-accelerations corresponded to forward/backward desired movements and x-accelerations corresponded to left/right desired movements. From this, it was determined that K-means was a preferable algorithm for tagging data, since it can form effective clusters while relying on fewer assumptions. With hardcoded initial clusters, the class labels for each direction are the same each time making the data consistent for testing.

Our methods were trained and tested on datasets each containing 6000 samples collected from the same session. On the test set our perceptron/softmax method achieved an accuracy of 71.1%, while the neural network achieved an accuracy of 73.5%. The final predictions for each method are shown in Figure 4 along with the ground truth. The two methods have comparable accuracy within 2.5%, although their predictions by class vary much more. When looking at each class individually in Figure 3 we see performance by class both between methods and within each method are not uniform. The neutral class seems to be the hardest to predict accurately. It was the true class for nearly 40% of incorrect predictions for the neural network. While the perceptron/softmax method only saw it make up about 20% of incorrect predictions, this method also predicted the neutral class 35% of the time it was incorrect - essentially predicting it correctly more often, but also predicting it incorrectly more often.

The around 70% accuracy and additional troubles with the neutral class may be a result of especially noisy data. During our work we found that training and testing on separate datasets that were collected from the same session gave much better results than using a test set that was from a different day of data collection. This suggests the output of the data sensors on the muscles may be influenced by other factors other than the arm movement itself, such as exact placement of the sensors or physical differences in the muscle.

VI. FUTURE WORK

While the trained predictor has demonstrated a capacity to effectively distinguish between different arm movements based

	Neutral	Down	Back	Forward	Up
Truth (PS)	20.54	25.90	15.00	17.77	20.77
Pred (PS)	35.37	33.35	14.08	8.48	8.71
Truth (NN)	39.96	21.84	10.70	12.15	15.36
Pred (NN)	27.88	31.34	16.68	9.31	14.79

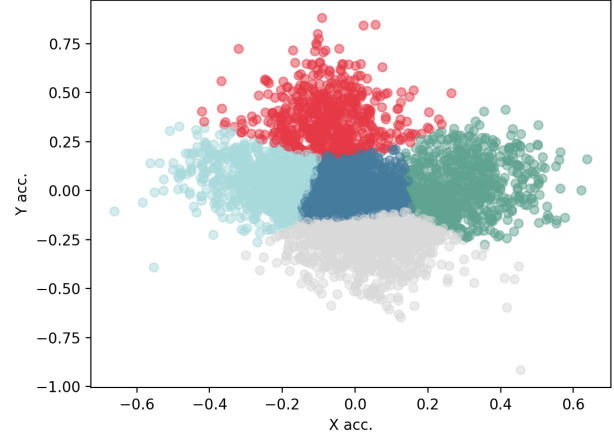
Fig. 3. This table shows the class breakdown by percentage for all incorrect predictions. The ‘Truth’ row shows what percentage of incorrect predictions had the given true class, and the ‘Pred’ row shows what percentage of incorrect predictions had the given predicted class. The PS is for the Perceptron/Softmax method, and the NN is for the Neural Network method.

on signals from the muscles, there are still several avenues for further development, both for refining the techniques used and for generalizing the results. For generalizing the results, in particular, it is possible to compile data from a variety of users; although the training and test data used was all generated specifically for the project, physical distance between team members and the hardware required for data collection restricted the team to collect data from a single team member. In particular, collecting data from users of multiple body types, backgrounds and age groups might help determine if a single configuration for control can be generated after gathering sufficient data from many users, if several different configurations should be generated for different body types, or if it is necessary to train the system separately for each user to maintain results of the precision seen in this project. Another potential area of improvement is, perhaps obviously, the addition of more muscle sensors. In addition to giving more precision when predicting on the movements seem in this proeject, it might also allow for the prediction of more types of movements, such as rotation of the end effector (measurable with a gyroscope, which was not included in this application).

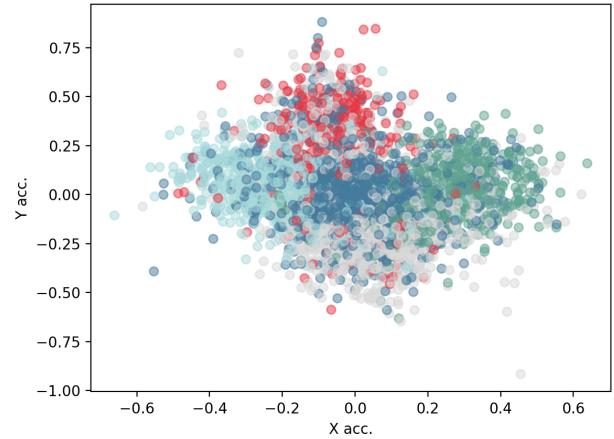
Most importantly, it is also possible to explore other forms of measurement, as well as hybrid approaches, in the generation of input data. For example, the relatively higher accuracy seen in softmax regression for categorization of directional information when compared to the perceptron used to compare the intensity of movement can be leveraged by using a switch to activate and deactivate limb movement (although this makes the interface more complicated). In addition, the use of hybrid data inputs would simplify direct control of an end effector; a switch or similar input can be used to actuate an end effector like a gripper or another type of tool, depending on the application. Similar methods can also be used to control a device, virtual or real, from other measurements taken from a person; as shown in the related works referenced in this report, EEG signals show a great deal of promise [6], but the equipment used to perform these measurements is for now less accessible than electromyographic sensors.

VII. CODE

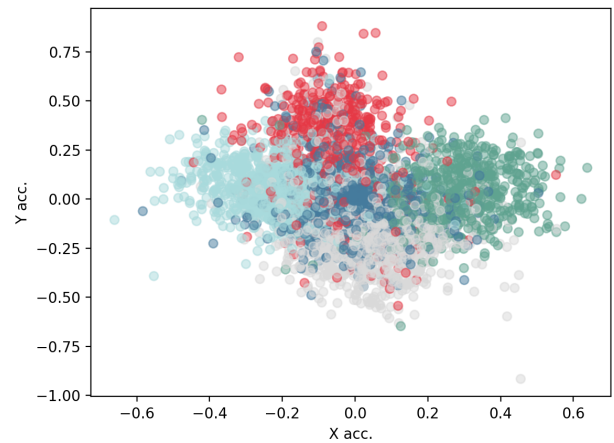
All code used in this project can be found in the github repository at <https://github.com/rae018/stanford-cs229>. Most of the code in the repository is in Python, but the code used to capture data from the muscle sensors and accelerometer is Arduino-based (a variant of C++).



(a) Ground Truth for the test set



(b) Perceptron/Softmax predictions on the test set



(c) Neural network predictions on the test set

Fig. 4. Results on the test set.

VIII. CONTRIBUTIONS

- 1) Leopoldo Contreras built all of the hardware required for data collection and wrote the code used by the microcontroller to record the accelerometer and muscle sensor data, as well as the code used to preprocess the input data to calculate the approximate derivatives and other signals. He collected all of the data used in this project, co-designed the method used to label data (including the use of K-means to do so) and co-designed the perceptron/softmax version of the predictor with the other team members.
- 2) Preston Rogers developed the method of data preprocessing necessary for K-means to effectively classify accelerometer data and wrote the code used for this processing and categorization (he also implemented the Mixture of Gaussians labeling method in early experiments). He co-designed the softmax/perceptron predictor with the other team members and implemented the use of Newton's Method for determining robot joint positions from the end effector position.
- 3) Reed Evans was the principal programmer for the perceptron/softmax predictor (which he co-designed with the other team members), and designed and implemented the neural net version of the predictor, including the training of each. Additionally, he implemented the graphing methods used to represent the data distributions seen in this report, and the methods used to divide data into training and test sets and calculate the accuracy of each predictor on each set.

REFERENCES

- [1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. USA: Cambridge University Press, 1st ed., 2017.
- [2] M. Bélanger-Barrette. <https://robohub.org/how-many-axes-does-my-robot-need/>, 2015.
- [3] K. Hartman. <https://cdn-learn.adafruit.com/downloads/pdf/getting-started-with-myoware-muscle-sensor.pdf>, 2016.
- [4] A. Van Boxtel, "Optimal signal bandwidth for the recording of surface emg activity of facial, jaw, oral, and neck muscles," *Psychophysiology*, vol. 38, no. 1, pp. 22–34, 2001.
- [5] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*. 2020. <https://d2l.ai>.
- [6] B. J. Edelman, J. Meng, D. Suma, C. Zurn, E. Nagarajan, B. Baxter, C. C. Cline, and B. He, "Noninvasive neuroimaging enhances continuous neural tracking for robotic device control," *Science robotics*, vol. 4, no. 31, 2019.
- [7] M. Burdis, "Plotting serial port data in real time using python and matplotlib," Feb 2019.
- [8] A. K. Tanwani, A. Billard, *et al.*, "Rewards-driven control of robot arm by decoding eeg signals," in *2014 36th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, pp. 1658–1661, Ieee, 2014.
- [9] P. K. Artemiadis and K. J. Kyriakopoulos, "Emg-based control of a robot arm using low-dimensional embeddings," *IEEE Transactions on Robotics*, vol. 26, no. 2, pp. 393–398, 2010.
- [10] H. F. Hassan, S. J. Abou-Loukh, and I. K. Ibraheem, "Teleoperated robotic arm movement using electromyography signal with wearable myo armband," *Journal of King Saud University - Engineering Sciences*, vol. 32, no. 6, pp. 378 – 387, 2020.
- [11] D. Dziuba, "Robotic hand control using emg," Jul 2019.
- [12] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. Software available from tensorflow.org.
- [13] "NumPy," 2019. Software available from numpy.org.